# DATABASE MANAGEMENT SYSTEM (CT 652)

*Tribhuvan University | Institute of Engineering | BCT | Year III / Part II*

## CHAPTER 3: Relational Languages

Syllabus Hours: 7 hrs | Marks Weight: 12 marks

*Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)*

### CHAPTER 3 SYLLABUS TOPICS


- **3.1** Introduction to SQL
- **3.2** Features of SQL
- **3.3** Queries and Sub-Queries
- **3.4** Set Operations
- **3.5** Relations — Joined and Derived
- **3.6** Queries under DDL and DML Commands
- **3.7** Embedded SQL
- **3.8** Views
- **3.9** Relational Algebra
- **3.10** Database Modification
- **3.11** QBE and Domain Relational Calculus


## SECTION 1: SQL — Introduction, Features & DDL/DML Commands

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Magh, 2072 Ashwin, 2070 Magh*

**Q1.** What is SQL? Explain the features and advantages of SQL. What are the different types of SQL commands with examples?

**[2+4]**
*2081 Chaitra 2078 Chaitra 2075 Bhadra*

**Answer:** Introduction to SQL

SQL (Structured Query Language) is the standard language for creating, managing, and querying relational databases. It was developed by IBM in the 1970s (originally called SEQUEL) and standardized by ANSI/ISO. SQL allows users to define database structures, manipulate data, and control access — all within the relational model.

**Key Features of SQL:**

| Feature | Description |
|---|---|
| Non-Procedural | User specifies WHAT data is needed, not HOW to retrieve it. The DBMS optimizer decides the execution plan. |
| Set-Oriented | SQL operates on entire sets of rows at once — not row-by-row like procedural languages. |

| | |
|---|---|
| ANSI/ISO Standardized | Supported by all major RDBMS: MySQL, PostgreSQL, Oracle, SQL Server, SQLite. |
| Data Independence | SQL works at the logical level — users do not need to know physical storage details. |
| Easy Syntax | English-like syntax makes SQL readable and learnable compared to low-level alternatives. |
| Supports DDL, DML, DCL, TCL | A complete language covering structure definition, data manipulation, access control, and transactions. |
| Embedded SQL | SQL statements can be embedded in host languages (C, Java, Python) for full application development. |
| View Support | SQL supports virtual tables (views) that simplify complex queries and provide data security. |
| Integrity Constraints | Enforces PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK automatically at the engine level. |
| Transaction Support | COMMIT, ROLLBACK, SAVEPOINT ensure atomic, consistent, isolated, and durable operations. |

**Types of SQL Commands:**

| Category | Full Form | Commands | Purpose |
|---|---|---|---|
| DDL | Data Definition Language | CREATE, ALTER, DROP, TRUNCATE | Define and modify database structure (tables, indexes, schemas). |
| DML | Data Manipulation Language | SELECT, INSERT, UPDATE, DELETE | Add, retrieve, update, and remove data in tables. |
| DCL | Data Control Language | GRANT, REVOKE | Control user access permissions. |
| TCL | Transaction Control Language | COMMIT, ROLLBACK, SAVEPOINT | Manage database transactions safely. |

**Q2.** Write SQL statements to CREATE the given relational schema with appropriate PRIMARY KEY and FOREIGN KEY constraints. Then write SQL queries as specified.

**[2+6]**
*2081 Ashwin 2079*
*Chaitra 2075 Bhadra*

**Answer:** DDL — Creating Tables with Constraints

**Schema: Employee / Project / Work / Payment**

```
Employee (empid, ename, address, title, headid)
```

| | |
|---|---|
| Project | (pid, pname, budget, location) |
| Work | (empid, pid, responsibility, duration) |
| Payment | (title, salary) |

## SQL: CREATE TABLE Statements

```
CREATE TABLE Payment (
    title    VARCHAR(50)    PRIMARY KEY,
    salary   DECIMAL(10,2)  NOT NULL
);

CREATE TABLE Employee (
    empid    INT            PRIMARY KEY,
    ename    VARCHAR(100)   NOT NULL,
    address  VARCHAR(200),
    title    VARCHAR(50),
    headid   INT,
    FOREIGN KEY (title)  REFERENCES Payment(title),
    FOREIGN KEY (headid) REFERENCES Employee(empid)
);

CREATE TABLE Project (
    pid      INT            PRIMARY KEY,
    pname    VARCHAR(100)   NOT NULL,
    budget   DECIMAL(12,2),
    location VARCHAR(100)
);

CREATE TABLE Work (
    empid          INT,
    pid            INT,
    responsibility VARCHAR(100),
    duration       DECIMAL(5,2),
    PRIMARY KEY (empid, pid),
    FOREIGN KEY (empid) REFERENCES Employee(empid),
    FOREIGN KEY (pid)   REFERENCES Project(pid)
);
```

**Q3.** Write SQL queries for the Employee/Project/Work/Payment schema: (i) Count projects with duration > 2 years. (ii) Name of employees in ICT project earning > 20K. (iii) Update salary by 15% if < 50K, 10% if 50K–100K, 5% if > 100K. (iv) Create a view with empid, name, project title, budget.

**[4×2]**
*2081 Chaitra 2077*
*Chaitra 2076 Baisakh*

**Answer:** Employee/Project SQL Queries

**(i) Count number of projects with duration more than 2 years:**

```
SELECT COUNT(DISTINCT w.pid) AS project_count
FROM Work w
WHERE w.duration > 2;
```

**(ii) Names of employees working in ICT project and earning more than 20K:**

```
SELECT e.ename
FROM Employee e
JOIN Work w      ON e.empid  = w.empid
JOIN Project p   ON w.pid    = p.pid
JOIN Payment pay ON e.title  = pay.title
WHERE p.pname = 'ICT'
AND   pay.salary > 20000;
```

**(iii) Update salary — 15% if < 50K, 10% if 50K–100K, 5% if > 100K:**

```
UPDATE Payment
SET salary = CASE
    WHEN salary < 50000                  THEN salary * 1.15
    WHEN salary BETWEEN 50000 AND 100000 THEN salary * 1.10
    ELSE                                      salary * 1.05
END;
```

**(iv) Create a view — empid, name, project title, budget:**

```
CREATE VIEW EmpProjectView AS
    SELECT e.empid, e.ename, p.pname AS project_title, p.budget
    FROM Employee e
    JOIN Work w    ON e.empid = w.empid
    JOIN Project p ON w.pid   = p.pid;
```

## SECTION 2: SELECT Queries, Sub-Queries & Set Operations

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Baishakh, 2078 Poush, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Magh, 2071 Bhadra*

**Q4.** Consider the Library database schema below. Write SQL for: (i) Members who issued a book on Jan 20 2025. (ii) Librarian name who issued a title starting with "D" using sub-query. (iii) Update all librarian salaries — 10% raise if < 50000, 5% to the rest. (iv) Create a view BOOKMANAGER (BID, Title, LibID, Name) for Wiley books published 1990–2010.

**[4×2]**

*2081 Chaitra 2078 Poush 2076 Baisakh*

**Answer:** Library Database — SQL Queries

**Schema:**

| | |
|---|---|
| Book | (BID, Title, Author, Genre, Publisher, Year) |
| Member | (MID, Name, Email, Phone, Address) |
| IssuedBook | (LibID, MID, BID, IssueDate, ReturnDate) |
| Librarian | (LibID, Name, Email, Phone, Salary) |
| Manages | (LibID, BID) |

### (i) Display members who issued a book on Jan 20, 2025:

```
SELECT m.Name
FROM Member m
JOIN IssuedBook ib ON m.MID = ib.MID
WHERE ib.IssueDate = '2025-01-20';
```

### (ii) Librarian name who issued a book with title starting with "D" (sub-query):

```
SELECT l.Name
FROM Librarian l
WHERE l.LibID IN (
    SELECT ib.LibID
    FROM IssuedBook ib
    JOIN Book b ON ib.BID = b.BID
    WHERE b.Title LIKE 'D%'
);
```

### (iii) Update salaries — 10% raise if salary < 50000, else 5%:

```
UPDATE Librarian
SET Salary = CASE
    WHEN Salary < 50000 THEN Salary * 1.10
    ELSE                     Salary * 1.05
END;
```

### (iv) Create view BOOKMANAGER (Wiley books, Year 1990–2010):

```
CREATE VIEW BOOKMANAGER AS
    SELECT b.BID, b.Title, l.LibID, l.Name AS LibrarianName
    FROM Book b
    JOIN Manages m   ON b.BID   = m.BID
    JOIN Librarian l ON m.LibID  = l.LibID
    WHERE b.Publisher = 'Wiley'
    AND   b.Year BETWEEN 1990 AND 2010;
```

**Q5.** Consider the Accident database. Write SQL for: (i) Names and addresses of drivers who own a car and were in an accident. (ii) Total damage per accident report. (iii) Reports where damage exceeded Rs. 50,000. (iv) All cars in accidents with owner names.

**[4×2]**

*2082 Kartik 2079*
*Chaitra 2073 Bhadra*

**Answer:** Accident / Insurance Database — SQL Queries

**Schema:**

| | |
|---|---|
| Person | (driver_id, name, address) |
| Accident | (report_no, date, location) |
| Owns | (driver_id, car) |
| Participated | (driver_id, car, report_no, damage_amount) |

### (i) Names and addresses of drivers who own a car AND were in an accident:

```
SELECT DISTINCT p.name, p.address
FROM Person p
JOIN Owns o          ON p.driver_id = o.driver_id
JOIN Participated pa ON p.driver_id = pa.driver_id;
```

### (ii) Total damage cost for each accident report:

```
SELECT report_no, SUM(damage_amount) AS total_damage
FROM Participated
GROUP BY report_no;
```

### (iii) Accident reports where total damage exceeded Rs. 50,000:

```
SELECT report_no, SUM(damage_amount) AS total_damage
FROM Participated
GROUP BY report_no
HAVING SUM(damage_amount) > 50000;
```

### (iv) All cars involved in accidents with their owner names:

```
SELECT pa.car, p.name AS owner_name
FROM Participated pa
JOIN Owns o   ON pa.car       = o.car
JOIN Person p ON o.driver_id  = p.driver_id;
```

**Q6.** What is a Sub-Query (Nested Query)? Explain with examples. Differentiate between correlated and non-correlated sub-queries with SQL examples.

**[2+4]**
*2078 Baishakh 2076 Baisakh 2074 Bhadra*

**Answer:** Sub-Queries in SQL

A sub-query (nested query / inner query) is a SELECT statement embedded inside another SQL statement. The inner query executes first and its result is used by the outer query as a condition or value.

| Type | Definition | Execution | Example Use |
|---|---|---|---|
| Non-Correlated Sub-query | Inner query is completely independent of the outer query. | Executes ONCE; result passed to outer query. | Find employees earning more than the overall average salary. |
| Correlated Sub-query | Inner query references columns of the outer query — depends on outer row. | Executes ONCE PER ROW of the outer query. | Find employees earning more than their department average. |

**Sub-query Operators:**

| Operator | Meaning | Example |
|---|---|---|
| IN | Value matches any value in sub-query result set. | WHERE dept IN (SELECT dept FROM Dept WHERE city='KTM') |
| NOT IN | Value does not match any value in sub-query result. | WHERE eid NOT IN (SELECT eid FROM Work WHERE pid=5) |
| EXISTS | True if sub-query returns at least one row. | WHERE EXISTS (SELECT 1 FROM Work w WHERE w.empid=e.empid) |
| NOT EXISTS | True if sub-query returns no rows. | WHERE NOT EXISTS (SELECT 1 FROM Booking b WHERE b.pid=p.pid) |
| = (scalar) | Sub-query returns exactly one value. | WHERE salary = (SELECT MAX(salary) FROM Payment) |
| > ANY / ALL | Comparison with sub-query set. | WHERE salary > ALL (SELECT salary FROM Payment WHERE title='Jr') |

**Example 1 — Non-Correlated (find librarian who issued books with title starting D):**

```
SELECT l.Name
FROM Librarian l
WHERE l.LibID IN (
    SELECT ib.LibID
    FROM IssuedBook ib JOIN Book b ON ib.BID = b.BID
```

```
    WHERE b.Title LIKE 'D%'   -- inner query independent of outer
);
```

**Example 2 — Correlated (employees earning more than their department average):**

```
SELECT e.ename, e.title
FROM Employee e
JOIN Payment p ON e.title = p.title
WHERE p.salary > (
    SELECT AVG(p2.salary)
    FROM Payment p2
    JOIN Employee e2 ON e2.title = p2.title
    WHERE e2.headid = e.headid   -- references outer query column
);
```

**Q7.** Explain SQL Set Operations — UNION, INTERSECT, and EXCEPT — with examples and their rules. How are they different from JOINs?

**[2+4]**
*2078 Chaitra 2076*
*Baisakh 2070 Magh*

**Answer:** SQL Set Operations

SQL set operations combine the results of two or more SELECT statements. All participating SELECT statements must be union-compatible: same number of columns with compatible data types.

| Operation | Symbol | Returns | Duplicates |
|---|---|---|---|
| UNION | ∪ | All rows from BOTH queries combined. | Removes duplicates |
| UNION ALL | ∪ (all) | All rows from both queries, including duplicates. | Keeps duplicates |
| INTERSECT | ∩ | Only rows appearing in BOTH query results. | Removes duplicates |
| EXCEPT / MINUS | − | Rows in first query but NOT in second. | Removes duplicates |

**Rules for Set Operations:**

- Both SELECT statements must return the same number of columns.
- Corresponding columns must have compatible (not necessarily identical) data types.
- Column names in the result come from the FIRST SELECT statement.
- ORDER BY can only appear ONCE at the very end.

**Examples (Sailor / Boat / Reserves schema):**

```
sailor   (sailorid, sname, rating, age)
boat     (boatid, boatname, color)
```

```
reserves (sailorid, boatid, date)
```

```
-- UNION: sailors who reserved a red OR green boat
SELECT s.sname FROM sailor s JOIN reserves r ON s.sailorid=r.sailorid
                             JOIN boat b ON r.boatid=b.boatid WHERE
b.color='red'
UNION
SELECT s.sname FROM sailor s JOIN reserves r ON s.sailorid=r.sailorid
                             JOIN boat b ON r.boatid=b.boatid WHERE
b.color='green';

-- INTERSECT: sailors who reserved BOTH red AND green boats
... WHERE b.color='red' INTERSECT ... WHERE b.color='green';

-- EXCEPT: sailors who reserved a red boat but NOT a green boat
... WHERE b.color='red' EXCEPT ... WHERE b.color='green';
```

## SECTION 3: SQL Joins — Joined and Derived Relations

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2079 Ashwin, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra*

**Q8.** What are the different types of JOINs in SQL? Explain INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, NATURAL JOIN, and SELF JOIN with examples.

**[2+6]**
*2079 Ashwin 2077 Chaitra 2073 Bhadra*

**Answer:** SQL Joins — Types and Examples

A JOIN combines rows from two or more tables based on a related column (usually a primary–foreign key relationship). Different join types control which rows are included when there is no match between tables.

| Join Type | Returns | NULL Padding | Use Case |
|---|---|---|---|
| INNER JOIN | Only matching rows from BOTH tables. | No NULLs | Find records with data on both sides. |
| LEFT OUTER JOIN | ALL rows from LEFT + matching from RIGHT. No match → NULL for right columns. | Right side NULLs | See all left-side records even without a match. |
| RIGHT OUTER JOIN | ALL rows from RIGHT + matching from LEFT. No match → NULL for left columns. | Left side NULLs | See all right-side records even without a match. |
| FULL OUTER JOIN | ALL rows from BOTH tables. NULLs where no match on either side. | Both sides may have NULLs | Complete picture of both tables. |

| | | | |
|---|---|---|---|
| NATURAL JOIN | Joins on ALL columns with the same name; removes duplicate columns. | No NULLs | Tables share column names for join key. |
| CROSS JOIN | Cartesian product — every row of A paired with every row of B. | No NULLs | Combinatorial problems, test data. |
| SELF JOIN | A table joined with itself — used for hierarchical/recursive data. | No NULLs | Employee-manager, org chart. |

**SQL Examples (Employee / Department):**

```
-- INNER JOIN: only employees WITH a department
SELECT e.ename, d.dept_name
FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;

-- LEFT JOIN: ALL employees (even those with no department)
SELECT e.ename, d.dept_name
FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;

-- SELF JOIN: show each employee and their supervisor name
SELECT e.ename AS employee, m.ename AS supervisor
FROM Employee e JOIN Employee m ON e.headid = m.empid;

-- NATURAL JOIN: join on all matching column names automatically
SELECT * FROM Employee NATURAL JOIN Department;
```

## SECTION 4: Relational Algebra

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2078 Baishakh, 2078 Poush, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2075 Baisakh, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh*

**Q9.** What is Relational Algebra? Explain the fundamental operations of relational algebra with symbols, syntax, and examples. **[2+6]** *2079 Chaitra 2078 Baishakh 2074 Bhadra*

**Answer:** Relational Algebra

Relational Algebra is a procedural query language that operates on relations (tables) and produces a new relation as output. It forms the theoretical foundation of SQL. There are two categories of operations: basic (fundamental) and derived (built from basic ones).

| Operation | Symbol | Syntax | Description |
|---|---|---|---|
| Select (Restrict) | σ | σ_condition(R) | Returns tuples (rows) from R that satisfy the condition. Horizontal filtering. Equivalent to SQL WHERE clause. |

| | | | |
|---|---|---|---|
| Project | π | π_attr1,attr2(R) | Returns only the specified attributes (columns) from R. Vertical filtering. Equivalent to SQL SELECT. |
| Union | ∪ | R ∪ S | All tuples in R or S or both. R and S must be union-compatible (same schema). |
| Intersection | ∩ | R ∩ S | Only tuples that appear in BOTH R and S. R and S must be union-compatible. |
| Set Difference | − | R − S | Tuples in R but NOT in S. R and S must be union-compatible. |
| Cartesian Product | × | R × S | Every possible combination of tuples from R and S. Size = \|R\| × \|S\|. |
| Natural Join | □ | R □ S | Join on all common attribute names; duplicate columns removed automatically. |
| Theta Join | □_θ | R □_θ S | Join with any comparison condition θ (e.g., R.age > S.age). |
| Rename | ρ | ρ_name(R) | Renames a relation or its attributes. Required for self-joins. |
| Division | ÷ | R ÷ S | Returns tuples in R that are associated with ALL tuples in S. Used for "for all" queries. |
| Assignment | ← | T ← R | Assigns a relational algebra expression to a temporary variable T for reuse. |

**Most Repeated:** "Explain relational algebra operations with example" appears almost every year. Know all operations with symbols, definitions, and simple examples. Division (÷) and its use for "for all" queries is a favourite tricky question.

**Union-Compatibility Rule:**

For UNION (∪ ), INTERSECTION (∩), and SET DIFFERENCE (−), the two relations R and S must be union-compatible: same number of attributes AND corresponding attributes must have the same domain (data type).

**Q10.** Consider the University schema (Student, Course, Enrollment, Department, Professor, Teaches). Write Relational Algebra expressions for: (i) SID and name of students enrolled in "Database Systems". (ii) CID and title of CS courses with credits > 4. (iii) Increase salary of Mathematics professors by 15%.

**[3×2]**

*2082 Kartik 2080*
*Ashwin 2078 Chaitra*

**Answer:** University Schema — Relational Algebra

**Schema:**

```
Student     (sid, name, age, major, dept_id)
Course      (cid, title, credits, dept_id)
```

| |
|---|
| Enrollment (sid, cid, semester, year, grade) |
| Department (dept_id, dept_name, hod_id) |
| Professor (pid, name, salary, dept_id) |
| Teaches (pid, cid, semester, year) |

**(i) SID and name of all students enrolled in "Database Systems":**

```
π_sid,name ( Student ⋈ Enrollment ⋈ σ_title="Database Systems"(Course) )

-- Step-by-step:
DBCourse  ← σ_title="Database Systems" (Course)
Enrolled  ← DBCourse ⋈ Enrollment
Result    ← π_sid,name (Enrolled ⋈ Student)
```

**(ii) CID and title of CS courses with credits > 4:**

```
CSdept     ← σ_dept_name="Computer Science" (Department)
HighCredit ← σ_credits>4 (Course)
π_cid,title ( HighCredit ⋈ CSdept )
```

**(iii) Increase salary of Mathematics professors by 15%:**

```
MathDept    ← σ_dept_name="Mathematics" (Department)
MathProfs   ← Professor ⋈ MathDept

-- Remove old records, add updated ones:
Professor ← (Professor − MathProfs)
          ∪ π_pid,name,(salary×1.15),dept_id (MathProfs)
```

**Q11.** Consider the Library schema. Write Relational Algebra for: (i) Book title, author, and year where genre is Novel. (ii) Delete all members from Kathmandu City. (iii) Book title and librarian name for books published by Pearson.

**[3×2]**
*2081 Chaitra 2079*
*Ashwin 2077 Chaitra*

**Answer:** Library Schema — Relational Algebra

**(i) Book title, author, and publication year where genre is Novel:**

```
π_Title,Author,Year ( σ_Genre="Novel" (Book) )
```

**(ii) Delete all member information of members from Kathmandu City:**

```
KTMMembers ← σ_Address="Kathmandu City" (Member)
Member     ← Member − KTMMembers
```

**(iii) Book title and librarian name for books published by Pearson:**

```
PearsonBooks ← σ_Publisher="Pearson" (Book)
π_Title,Name ( PearsonBooks ⋈ Manages ⋈ Librarian )
```

**Q12.** Consider the Insurance/Accident schema (Person, Car, Accident, Owns, Participated). Write Relational Algebra for: (i) All persons with license living in Lalitpur. (ii) Total accidents by location on Dec 10 2022. (iii) Person name, date, location of accidents with damage > 50000. (iv) Person name and car brand manufactured in 2020.

**[4×2]**
*2080 Ashwin 2078*
*Baishakh 2075 Bhadra*

**Answer:** Insurance Schema — Relational Algebra

**Schema:**

| | |
|---|---|
| PERSON | (licenseNo, name, address) |
| CAR | (modelNo, brand, year) |
| ACCIDENT | (reportNo, date, location) |
| OWNS | (licenseNo, modelNo) |
| PARTICIPATED | (licenseNo, reportNo, damageAmount) |

**(i) All persons who have a license and live in Lalitpur:**

```
π_licenseNo,name ( σ_address="Lalitpur" (PERSON) )
```

**(ii) Total number of accidents by location on Dec 10, 2022 (uses Aggregate operator G):**

```
location G COUNT(reportNo)→total_accidents
  ( σ_date="2022-12-10" (ACCIDENT) )
```

**(iii) Person name, date, and location where damage exceeds 50,000:**

```
HighDamage ← σ_damageAmount>50000 (PARTICIPATED)
π_name,date,location ( PERSON □ HighDamage □ ACCIDENT )
```

**(iv) Name of person and brand of car manufactured in 2020:**

```
Cars2020 ← σ_year=2020 (CAR)
π_name,brand ( PERSON □ OWNS □ Cars2020 )
```

## SECTION 5: Views and Database Modification

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2079 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra*

**Q13.** What are database views? What are the advantages of using views in a DBMS? Write SQL to create a view and query it.

**[1+3+2]**
*2081 Chaitra 2079 Chaitra 2077 Chaitra*

**Answer:** Views in SQL

A VIEW is a virtual table based on the result of a SQL SELECT query. The view itself does not store data physically — it stores only the query definition. Every time the view is accessed, the DBMS executes the underlying query to produce the result.

| Advantage | Explanation |
|---|---|
| Data Security / Access Control | Views expose only selected columns or rows. Sensitive data (e.g., salary) can be hidden from certain users. |
| Simplification / Abstraction | Complex multi-table joins can be saved as a view. Users query the simple view instead of writing complex SQL. |
| Logical Data Independence | If the underlying table structure changes, only the view definition is updated — user queries remain the same. |
| Customized User Perspective | Different users or departments can have different views of the same data tailored to their needs. |
| Consistent Data Presentation | Derived and calculated columns can be pre-defined in a view for consistent access across users. |

### SQL: Create and Query a View

```
-- View 1: BOOKMANAGER — books managed by librarians, published by Wiley,
year 1990-2010
CREATE VIEW BOOKMANAGER AS
    SELECT b.BID, b.Title, l.LibID, l.Name AS LibrarianName
    FROM Book b
    JOIN Manages m   ON b.BID  = m.BID
    JOIN Librarian l ON m.LibID = l.LibID
    WHERE b.Publisher = 'Wiley'
    AND   b.Year BETWEEN 1990 AND 2010;

-- Query the view exactly like a table:
SELECT * FROM BOOKMANAGER;
SELECT LibrarianName FROM BOOKMANAGER WHERE Title LIKE 'D%';
```

```
-- View 2: PERSON_REPORT — persons with damage <= 100000
CREATE VIEW PERSON_REPORT AS
    SELECT p.licenseNo, p.name, pa.reportNo, pa.damageAmount
    FROM Person p
    JOIN Participated pa ON p.licenseNo = pa.licenseNo
    WHERE pa.damageAmount <= 100000;

-- Drop a view:
DROP VIEW BOOKMANAGER;
```

**Q14.** Explain INSERT, UPDATE, and DELETE commands (Database Modification) in SQL with syntax and examples for the given schema.

**[2+4]**
*2079 Chaitra 2078*
*Baishakh 2074 Bhadra*

**Answer:** Database Modification Commands

**INSERT — Adding New Tuples:**

```
-- Syntax:
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);

-- Example 1: Insert a new employee
INSERT INTO Employee (empid, ename, address, title, headid)
VALUES (101, 'Ram Bahadur', 'Kathmandu', 'Engineer', NULL);

-- Example 2: Insert from SELECT (bulk insert)
INSERT INTO HighPayEmployee (empid, ename)
SELECT e.empid, e.ename
FROM Employee e JOIN Payment p ON e.title = p.title
WHERE p.salary > 60000;
```

**UPDATE — Modifying Existing Tuples:**

```
-- Syntax:
UPDATE table_name SET col1=val1, col2=val2 WHERE condition;

-- Example 1: 10% salary raise for all librarians
UPDATE Librarian SET Salary = Salary * 1.10;

-- Example 2: Conditional update using CASE
UPDATE Payment
SET salary = CASE
    WHEN salary < 50000 THEN salary * 1.15
    WHEN salary BETWEEN 50000 AND 100000 THEN salary * 1.10
    ELSE salary * 1.05
END;
```

## DELETE — Removing Tuples:

```
-- Syntax:
DELETE FROM table_name WHERE condition;

-- Example 1: Delete all members from Kathmandu
DELETE FROM Member WHERE Address = 'Kathmandu City';

-- Example 2: Delete doctors starting with M on 10th floor
DELETE FROM Doctor
WHERE name LIKE 'M%'
AND name IN (
    SELECT w.name FROM Works w
    JOIN Department d ON w.dept_no = d.dept_no
    WHERE d.floor = 10
);

-- TRUNCATE (delete all rows, faster, auto-committed):
TRUNCATE TABLE TempEmployee;
```

| Command | Can Use WHERE? | Can Rollback? | Triggers Fired? | Auto-Committed? |
|---|---|---|---|---|
| DELETE | YES | YES (DML) | YES | NO |
| TRUNCATE | NO (removes all) | NO (DDL) | Usually NO | YES |
| DROP | N/A (removes table) | NO (DDL) | NO | YES |

# SECTION 6: Embedded SQL, QBE and Relational Calculus

*Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2078 Baishakh, 2075 Baisakh, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2070 Magh*

**Q15.** What is Embedded SQL? Why is it needed? Show an example of Embedded SQL in a C program using a cursor for multi-row queries.

**[2+5]**
*2078 Baishakh 2074 Bhadra 2072 Ashwin*

**Answer:** Embedded SQL

Embedded SQL is the technique of writing SQL statements directly inside a host programming language such as C, C++, Java, or COBOL. This is needed because SQL alone is not a complete programming language — it lacks loops, conditionals, and file I/O needed for real applications.

| Limitation of SQL Alone | Solution via Embedded SQL |
|---|---|
| Cannot implement loops, if-else, or decision logic. | Host language (C, Java) provides procedural control flow. |
| Cannot build standalone programs or user interfaces. | Full applications with menus, reports, and file I/O become possible. |
| Cannot process results row-by-row (needed for reports). | Cursor mechanism fetches one row at a time into host variables. |
| Cannot handle complex error conditions. | Host language exception handling manages SQL errors via SQLCA. |

**Key Concepts:**

- EXEC SQL: Prefix for all embedded SQL statements in C/COBOL. The precompiler replaces them with function calls.
- SQLCA (SQL Communication Area): A data structure checked after each SQL call to detect errors and return codes.
- Cursor: A named pointer into a multi-row result set. Used to FETCH rows one at a time.
- Host Variables: C variables used inside SQL statements, prefixed with colon (:var_name).
- Precompiler: Converts embedded SQL source code into pure C before compilation (e.g., proc for Oracle, ecpg for PostgreSQL).

**Embedded SQL in C — Cursor Example:**

```
#include <stdio.h>
EXEC SQL INCLUDE SQLCA;          /* Include SQL Communication Area */

EXEC SQL BEGIN DECLARE SECTION;
```

```
    char  ename[50];
    float salary;
EXEC SQL END DECLARE SECTION;

void main() {
    EXEC SQL CONNECT TO mydb;    /* Connect to database */

    /* Declare a cursor for a multi-row query */
    EXEC SQL DECLARE emp_cursor CURSOR FOR
        SELECT ename, salary
        FROM Employee
        WHERE salary > 50000;

    EXEC SQL OPEN emp_cursor;    /* Execute the query */

    while (1) {
        /* Fetch one row into host variables */
        EXEC SQL FETCH emp_cursor INTO :ename, :salary;
        if (SQLCA.SQLCODE != 0) break;  /* No more rows */
        printf("Name: %s | Salary: %.2f\n", ename, salary);
    }

    EXEC SQL CLOSE emp_cursor;   /* Release cursor */
    EXEC SQL COMMIT;             /* Save any changes */
    EXEC SQL DISCONNECT;
}
```

**Q16.** What is QBE (Query By Example)? Write QBE skeleton tables for: (i) Employee names and address earning > 50K with name containing "a". (ii) Manager name starting with "M" and salary > 60,000. (iii) Sailors name and age who reserved a red boat.

**[2+4]**

*2081 Ashwin 2080*
*Ashwin 2079 Chaitra*

**Answer:** QBE — Query By Example

QBE (Query By Example) is a visual, table-based query interface developed by Moshe Zloof at IBM in 1977. Instead of writing SQL text, users fill in example values directly in skeleton (template) tables and the system derives the SQL query from those examples. QBE was the forerunner of modern GUI query tools.

**QBE Notation:**

| Symbol | Meaning |
|---|---|
| P. | Print — display this column in the result. |
| _x (underscore) | A variable — links the same value across different columns or tables (like a join condition). |

| | |
|---|---|
| Condition (e.g., >50000) | A filter written directly in the cell (like SQL WHERE). |
| LIKE "%a%" | Pattern matching — same as SQL LIKE operator. |
| I. | Insert a new tuple into this relation. |
| D. | Delete matching tuples from this relation. |
| U. | Update — modify the value in this cell. |

**(i) Employee names and address earning > 50,000 with name containing "a" (2072 Ashwin):**

| Employee | empid | ename | address | title | salary |
|---|---|---|---|---|---|
| | | P._n | P._a | _t | |
| Payment | | | | _t | >50000 |
| Condition: | | _n LIKE "%a%" | | | |

**(ii) Manager whose name starts with "M" and salary > 60,000:**

| Employee | empid | ename | title | headid |
|---|---|---|---|---|
| | | P._m | Manager | |
| Payment | | | _t | >60000 |
| Cond: | | _m LIKE "M%" | | |

**(iii) Sailors name and age who reserved a red boat:**

| sailor | sailorid | sname | rating | age |
|---|---|---|---|---|
| | _sid | P. | | P. |

| reserves | sailorid | boatid | date |
|---|---|---|---|
| | _sid | _bid | |

| boat | boatid | boatname | color |
|---|---|---|---|
| | _bid | | red |

**Q17.** What is Relational Calculus? Differentiate between Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC). Give one example of each.

**[2+5]**
*2078 Chaitra 2076*
*Baisakh 2074 Bhadra*

**Answer:** Relational Calculus

Relational Calculus is a NON-PROCEDURAL query language — the user describes WHAT data to retrieve without specifying HOW to retrieve it. Unlike Relational Algebra (which is procedural), Relational Calculus is based on mathematical predicate logic (first-order logic).

| Feature | Tuple Relational Calculus (TRC) | Domain Relational Calculus (DRC) |
|---|---|---|
| Variable Range | Variables range over TUPLES (complete rows). | Variables range over DOMAINS (individual attribute values). |
| Notation | {t \| condition(t)} | {<x1,x2,...> \| condition(x1,x2,...)} |
| Origin | Theoretical basis for SQL. | Theoretical basis for QBE. |
| Reference Style | t.attribute to access an attribute of a tuple. | Individual domain variables are directly listed. |
| Quantifiers | ∃ (there exists), ∀ (for all) over tuples. | ∃ (there exists), ∀ (for all) over domain values. |

**TRC Example — Find names and addresses of persons with damage ≤ 100,000 (Accident DB):**

```
{ t.name, t.address |
  □ pa □ Participated (
       pa.licenseNo = t.licenseNo
       ∧ pa.damageAmount ≤ 100000 )
  ∧ t ∈ Person }
```

**DRC Example — Find passengers with gender = "male" who booked through Jet Agency:**

```
{ <pid, pname, pgender, pbirthplace> |
  <pid, pname, pgender, pbirthplace> □ Passenger
  ∧ pgender = "male"
  ∧ ∃ aid, aname, acity (
        <aid, aname, acity> □ Agency
        ∧ aname = "Jet"
        ∧ ∃ fid, bookdate (<pid, aid, fid, bookdate> ∈ Booking)
  )
}
```

## SECTION 7: Additional SQL Practice — Hotel, Flight, Doctor, Worker

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Poush, 2077 Chaitra, 2076 Baisakh, 2075 Baisakh, 2073 Bhadra, 2072 Magh, 2070 Magh*

**Q18.** Consider the Hotel/Room/Booking/Guest schema. Write SQL for: (i) Guests who booked rooms at Everest Hotel. (ii) Create a view with Hotel_No, Guest_No, Room_No, and price. (iii) Total cost of deluxe rooms with 5% discount. (iv) Hotel name with highest total guests.

**[4×2]**
*2079 Ashwin 2077 Chaitra 2070 Magh*

**Answer:** Hotel Database — SQL Queries

**Schema:**

```
Hotel   (Hotel_No, Name, Address)
Room    (Room_No, Hotel_No, Type, Price)
Booking(Hotel_No, Guest_No, Date_From, Date_To, Room_No)
Guest   (Guest_No, Name, Address)
```

### (i) Guests who booked rooms at the Everest Hotel:

```
SELECT g.Name
FROM Guest g
JOIN Booking b ON g.Guest_No = b.Guest_No
JOIN Hotel h   ON b.Hotel_No = h.Hotel_No
WHERE h.Name = 'Everest Hotel';
```

### (ii) Create a view: Hotel_No, Guest_No, Room_No, Price of all booked rooms:

```
CREATE VIEW BookedRooms AS
    SELECT b.Hotel_No, b.Guest_No, b.Room_No, r.Price
    FROM Booking b
    JOIN Room r ON b.Room_No  = r.Room_No
              AND b.Hotel_No  = r.Hotel_No;
```

### (iii) Total cost of all Deluxe rooms in Everest Hotel with 5% discount:

```
SELECT SUM(r.Price * 0.95) AS total_discounted_cost
FROM Room r
JOIN Hotel h ON r.Hotel_No = h.Hotel_No
WHERE h.Name = 'Everest Hotel'
AND   r.Type = 'Deluxe';
```

### (iv) Hotel name with the highest total number of guests:

```
SELECT h.Name, COUNT(b.Guest_No) AS total_guests
```

```
FROM Hotel h
JOIN Booking b ON h.Hotel_No = b.Hotel_No
GROUP BY h.Hotel_No, h.Name
ORDER BY total_guests DESC
LIMIT 1;
```

**Q19.** Consider the Passenger/Agency/Flight/Booking schema. Write SQL for: (i) Passenger details from Kathmandu to Pokhara. (ii) Update booking amount with 10% discount where destination = birthplace. (iii) Create View EsewaReport. (iv) Flight-wise total bookings for current date.

**[4×2]**
*2078 Chaitra 2075*
*Bhadra 2073 Bhadra*

**Answer:** Flight / Agency Database — SQL Queries

**Schema:**

| | |
|---|---|
| Passenger | (pid, pname, pgender, pbirthplace) |
| Agency | (aid, aname, acity) |
| Flight | (fid, fdate, time, source, destination) |
| Booking | (pid, aid, fid, bookdate, amount) |

**(i) Passenger details travelling from Kathmandu to Pokhara:**

```
SELECT p.*
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Flight f  ON b.fid = f.fid
WHERE f.source      = 'Kathmandu'
AND   f.destination = 'Pokhara';
```

**(ii) Update booking amount with 10% discount if flight destination = passenger birthplace:**

```
UPDATE Booking b
SET amount = amount * 0.90
WHERE EXISTS (
    SELECT 1
    FROM Passenger p, Flight f
    WHERE p.pid = b.pid
    AND   f.fid = b.fid
    AND   f.destination = p.pbirthplace
);
```

**(iii) Create View EsewaReport — total booking amount for current date through Esewa:**

```
CREATE VIEW EsewaReport AS
    SELECT SUM(b.amount) AS total_amount, b.bookdate
    FROM Booking b
    JOIN Agency a ON b.aid = a.aid
```

```
    WHERE a.aname     = 'Esewa'
    AND   b.bookdate  = CURDATE()
    GROUP BY b.bookdate;
```

**(iv) Flight-wise total bookings for the current date:**

```
SELECT b.fid, COUNT(*) AS total_bookings
FROM Booking b
WHERE b.bookdate = CURDATE()
GROUP BY b.fid;
```

**Q20.** Consider the Doctor/Works/Department schema. Write SQL for: (i) Doctor records with department info. (ii) Total rooms on each floor. (iii) Doctor with maximum age. (iv) Delete doctors with name starting with M who work on the 10th floor.

**[4×2]**
*2079 Chaitra 2076*
*Baisakh 2072 Magh*

**Answer:** Doctor Database — SQL Queries

**Schema:**

| | |
|---|---|
| Doctor | (name, age, address) |
| Works | (name, dept_no) |
| Department | (dept_no, floor, room) |

**(i) Doctor records with department information:**

```
SELECT d.name, d.age, d.address, dep.dept_no, dep.floor, dep.room
FROM Doctor d
JOIN Works w      ON d.name    = w.name
JOIN Department dep ON w.dept_no = dep.dept_no;
```

**(ii) Total number of rooms on each floor:**

```
SELECT floor, COUNT(room) AS total_rooms
FROM Department
GROUP BY floor;
```

**(iii) Display the name of the doctor with the maximum age:**

```
SELECT name FROM Doctor
WHERE age = (SELECT MAX(age) FROM Doctor);
```

**(iv) Delete doctors whose name starts with "M" and work on 10th floor:**

```
DELETE FROM Doctor
WHERE name LIKE 'M%'
AND name IN (
    SELECT w.name FROM Works w
```

```
    JOIN Department dep ON w.dept_no = dep.dept_no
    WHERE dep.floor = 10
);
```

**Q21.** Consider the Salesman/Customer/Order schema. Write SQL for: (i) Customer names, salesman names and city who live in the same city. (ii) All customers with their salesman. (iii) Orders by customers NOT in the same city as their salesman. (iv) Total orders per salesman with their name.

**[4×2]**
*2078 Baishakh 2075*
*Baisakh 2071 Bhadra*

**Answer:** Salesman Database — SQL Queries

**Schema:**

| |
|---|
| Salesman (salesman_id, name, commission, city) |
| Customer (customer_id, cust_name, city, grade, salesman_id) |
| Order (order_id, purchase_amt, order_date, customer_id, salesman_id) |

**(i) Customer names, salesman names and city — same city:**

```
SELECT c.cust_name, s.name AS salesman_name, c.city
FROM Customer c
JOIN Salesman s ON c.salesman_id = s.salesman_id
WHERE c.city = s.city;
```

**(ii) All customers with the name of the salesman who works for them:**

```
SELECT c.cust_name, s.name AS salesman_name
FROM Customer c
LEFT JOIN Salesman s ON c.salesman_id = s.salesman_id;
```

**(iii) Orders by customers NOT in the same city as their salesman:**

```
SELECT o.*
FROM `Order` o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Salesman s ON o.salesman_id = s.salesman_id
WHERE c.city <> s.city;
```

**(iv) Total orders handled by each salesman with their name:**

```
SELECT s.name, COUNT(o.order_id) AS total_orders
FROM Salesman s
LEFT JOIN `Order` o ON s.salesman_id = o.salesman_id
GROUP BY s.salesman_id, s.name;
```

## QUICK REVISION: Chapter 3 — Key Points Summary

| Topic | Key Points | Expected Marks |
|---|---|---|
| SQL Introduction | Non-procedural, set-oriented, ANSI standard. DDL/DML/DCL/TCL — know all categories with all commands. | 2-4 |
| DDL | CREATE (PK, FK, NOT NULL, UNIQUE, CHECK), ALTER (ADD/MODIFY/DROP COLUMN), DROP, TRUNCATE — auto-committed. | 2-4 |
| SELECT Queries | SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY. Aggregate: COUNT,SUM,AVG,MIN,MAX. Always check HAVING vs WHERE. | 3-6 |
| Sub-queries | Non-correlated (runs once) vs Correlated (runs per row). Operators: IN, NOT IN, EXISTS, NOT EXISTS, ANY, ALL, =. | 2-4 |
| Set Operations | UNION/INTERSECT/EXCEPT — union-compatible (same # columns, compatible types). UNION removes dups; UNION ALL keeps. | 2-3 |
| Joins | INNER (match only), LEFT/RIGHT/FULL OUTER, NATURAL (match same-name cols), SELF (table with itself), CROSS (cartesian). | 3-6 |
| Views | CREATE VIEW AS SELECT. Virtual table — stores query, not data. Advantages: security, abstraction, independence. | 1+3 |
| Database Modification | INSERT (single/bulk), UPDATE (CASE for multi-condition), DELETE (WHERE). TRUNCATE vs DELETE vs DROP. | 2-4 |
| Relational Algebra | σ (select), π (project), ∪ ∩ − × ⋈ ρ ÷ — procedural, each operation returns a relation. | 4-8 |
| RA — Union Compat. | ∪ ∩ − require same # attributes with same domains. Natural join removes duplicate columns automatically. | 2-4 |
| Embedded SQL | SQL inside C/Java; EXEC SQL prefix; cursor for multi-row; :variable for host vars; SQLCA for error checking. | 2-5 |
| QBE | Visual table-based query; P.=print, _x=variable/link, conditions in cells. Zloof, IBM 1977. Basis for GUI tools. | 2-4 |
| Relational Calculus | Non-procedural; TRC={t\|cond(t)} for SQL; DRC={<x1,x2>\|cond} for QBE. Uses ∃ and ∀ quantifiers. | 2-4 |